

Classification Challenge

Author: Negar Fathi

Student ID: 400222069

1. Introduction

The mission in this competition is to build a model that predicts a 11 class category from 64 synthesized flags. The features mimic “present / absent” indicators such as abnormal lab values, symptoms, or billing codes, making the task surprisingly subtle despite the modest table size. This report is a detailed explanation on how I proceeded to tackle this challenge.

2. Data

2.1 Data Loading and Preview

First both the train and test datasets were loaded into a dataframe. We put the test dataframe to the side for now. We will work on train dataset to avoid leakage. Train dataset has 564 rows and 66 columns. The columns are ID, feature 0, feature2, ..., feature63 and label. All columns are integers. The feature columns look like they are encoded meaning the values of features are either 1 or zero. Also there are no null values.

2.2 EDA

To check the distribution of the classes a bar plot is created as seen in Figure 1. According to Figure 1 class label 8 has the highest number of samples reaching 70 and following closely behind it is class label 2. Class label 4 has the lowest number of samples with around 35 samples.

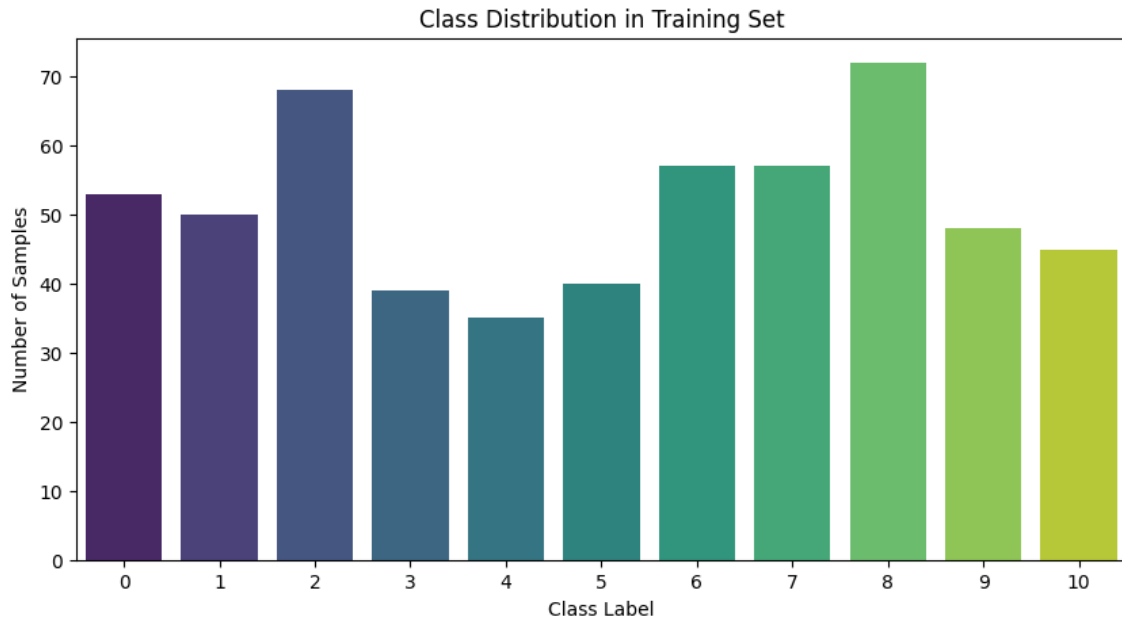


Figure 1

Next we have another barplot, this time to check the frequency of each feature or in other words the frequency of 1s. According to Figure 2 feature4 has the highest frequency and feature63 has the lowest frequency.

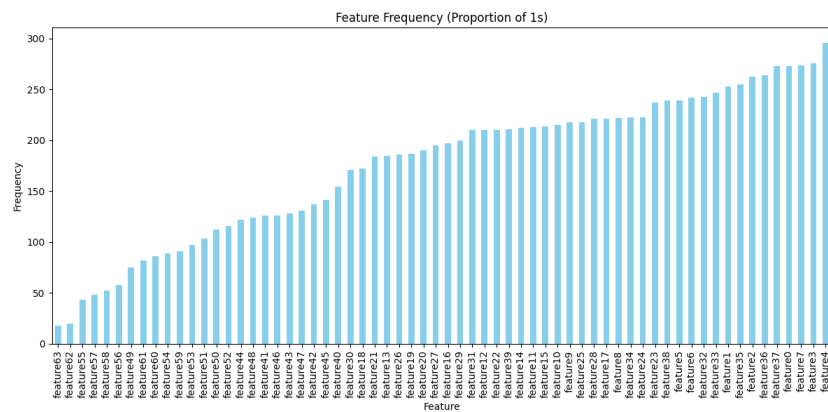


Figure 2

In Figure 3 we can see the mutual information between features and target using MI score. MI score is a measure of the statistical dependence between two random variables. It quantifies how much information one variable provides about another. A higher MI score indicates a stronger relationship between the variables. Here the top 5 informative features are feature37, feature35, feature60, feature61 and feature36.

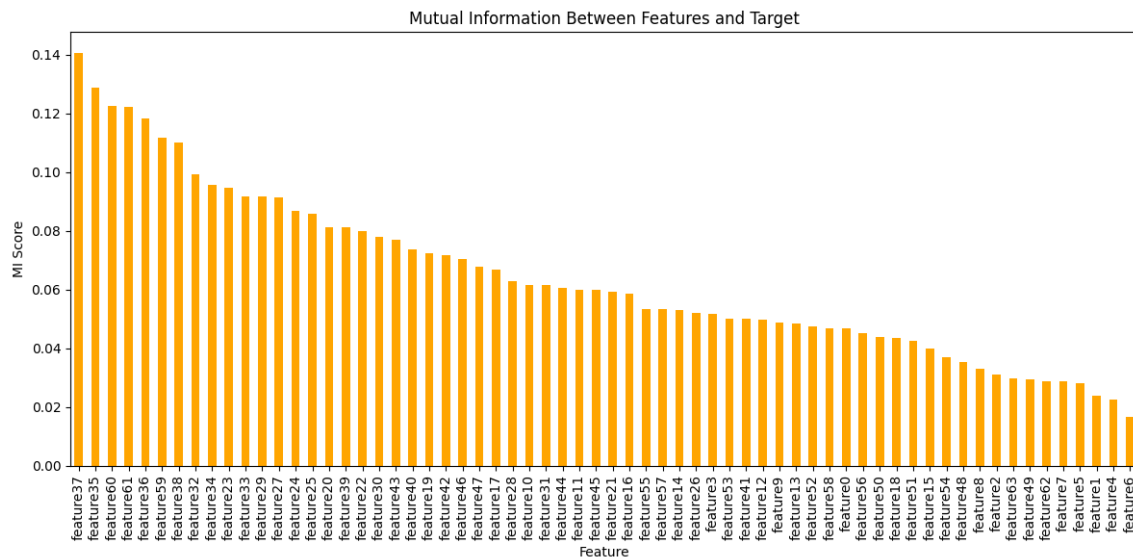


Figure 3

In the correlation heatmap in Figures 4 the noticeable thing is that there's a very strong positive correlation between feature55 and feature56 to feature 58.

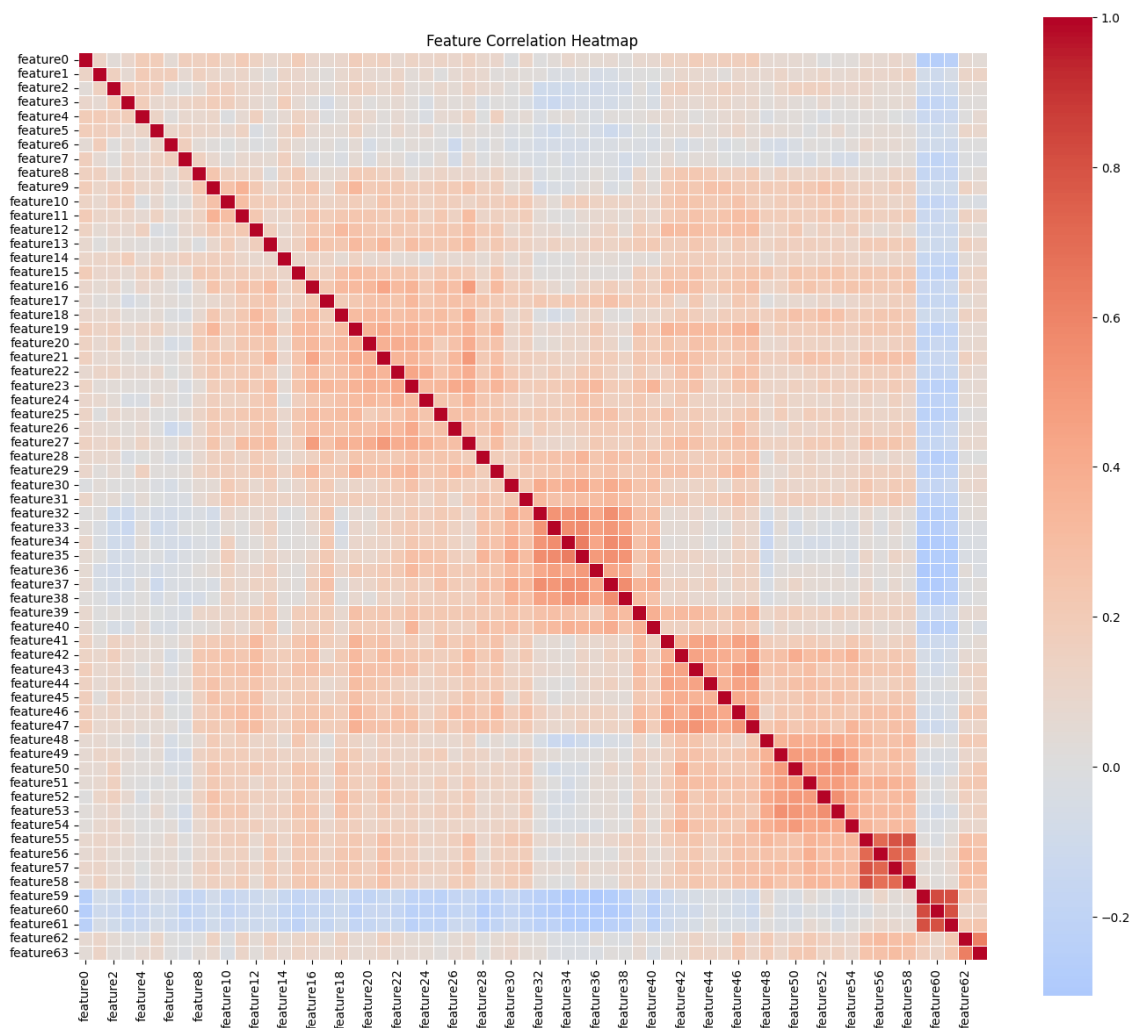


Figure 4

In Figure 5 and 6 we have the 2D PCA and 3D PCA respectively.

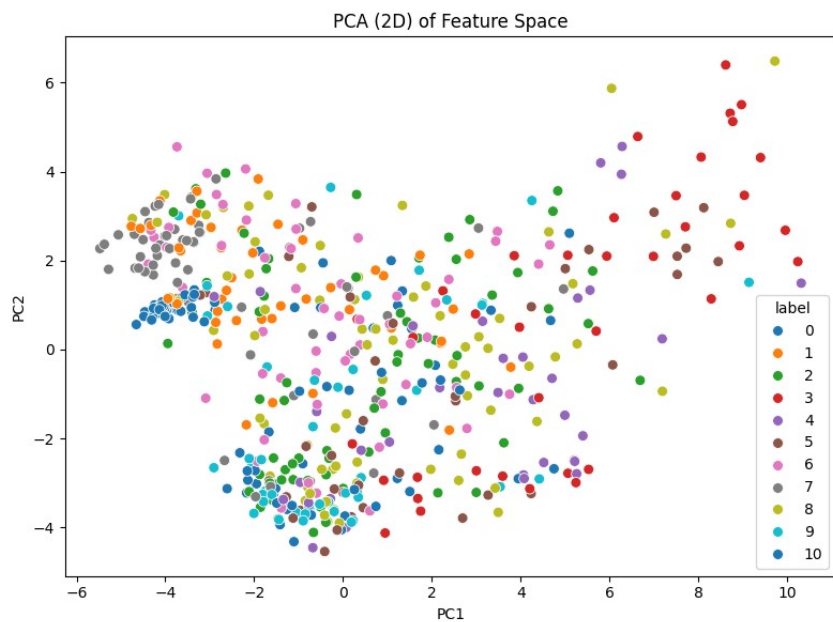


Figure 5

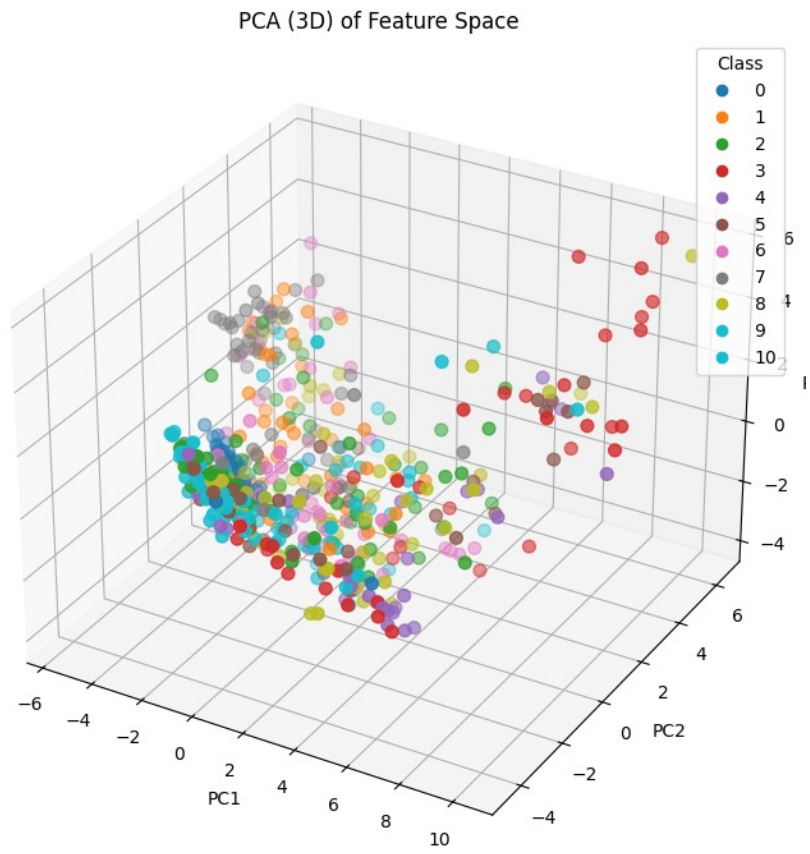


Figure 6

3. Baseline Models

We trained and evaluated 3 different baseline models: Logistic Regression, SVM with RBF kernel and Naive Bayes. Each model was trained using an 80/20 stratified split of the training dataset, ensuring that the label distribution remained consistent across training and testing sets. The features were scaled where necessary.

3.1 Logistic Regression

The Logistic Regression model was trained using the multinomial setting and the 'lbfgs' solver. Standard scaling was applied to the binary input features, even though they are already in a normalized format, in order to ensure compatibility with the solver and preserve consistency across models. This model achieved an accuracy of approximately 34.51% on the test split.

3.2 SVM with RBF Kernel

The SVM model was configured with an RBF kernel and used a one-vs-rest decision function. Like Logistic Regression, it also relied on scaled features. The SVM showed a slight improvement over Logistic Regression, reaching an accuracy of 37.17%. This suggests that the decision boundaries between classes are likely non-linear and benefit from the flexibility provided by the RBF kernel.

3.3 Naive Bayes

Interestingly, the Naive Bayes classifier outperformed both other baselines, with an accuracy of 38.05%. This is somewhat surprising given its simplifying assumption that all features are conditionally independent given the class label. However, it is well-suited for binary input features and small to medium-sized datasets, which aligns well with the nature of our data.

4. Ensemble Methods

4.1 Bagging

We next evaluated two popular ensemble methods based on bagging: Random Forest and Extra Trees. These models combine multiple decision trees to improve predictive performance and reduce variance.

4.1.1 Random Forest Classifier

The Random Forest classifier was configured with 200 estimators and default depth, using all features and training samples without restriction. Despite its reputation for strong baseline performance on tabular data, it achieved an accuracy of only 31.86% on the test set. This suggests that the diversity in tree splits did not significantly enhance

generalization over simpler models like Naive Bayes or SVM.

4.1.2 Extremely Randomize Trees

We also tested Extra Trees, an ensemble similar to Random Forest but with greater randomization in how splits are selected during training. Extra Trees yielded a slightly lower accuracy of 30.08%. The performance gap indicates that introducing more randomness may have hurt this model's ability to capture informative patterns in the data.

4.2 Boosting

Next, we explored boosting algorithms to see if they could outperform the models evaluated so far. These algorithms build models sequentially, each one correcting the errors of the previous, which often results in higher accuracy on structured datasets.

4.2.1 XGBoost

We first tested XGBoost, which is known for its speed and performance. It was run with 200 estimators, learning rate 0.1, and maximum depth 6. The resulting accuracy was 31.86%, the same as Random Forest. This indicates that out-of-the-box XGBoost struggled to capture the patterns in the dataset, perhaps due to the binary nature of the features or a need for deeper tuning.

4.2.2 LightGBM

Next, we tried LightGBM with similar hyperparameters. Surprisingly, it performed slightly worse, with an accuracy of 30.08%. LightGBM is usually very efficient on large and sparse datasets, but in this case, it did not extract meaningful patterns from the data.

4.2.3 CatBoost

Finally, we tested CatBoost, which tends to perform well on tabular data even without extensive tuning. It outperformed both XGBoost and LightGBM, achieving an accuracy of 36.28%. While still slightly behind Naive Bayes and SVM, CatBoost showed that boosting can provide value with minimal setup.

4.3 Ensemble Learning

To try and boost overall performance, we combined several models using ensemble strategies. The idea is that by aggregating multiple diverse classifiers, we can reduce individual weaknesses and improve generalization.

4.3.1 Soft Voting Classifier

We first implemented a soft voting classifier, which averages the predicted probabilities from several base models. We selected four strong individual models: Logistic Regression, SVM (RBF kernel), Naive Bayes, and CatBoost. The voting ensemble achieved an accuracy of 36.28%, which matched the performance of CatBoost and outperformed some of the other individual learners like Random Forest and LightGBM.

4.3.2 Hard Voting Classifier

After the soft voting classifier, we switched to a hard voting ensemble. In this approach, the final prediction is based on the majority vote from individual classifiers. We used the same set of models as before except for CatBoost: Logistic Regression, SVM (RBF kernel), Naive Bayes. This configuration resulted in an accuracy of 38.94%, making it the best-performing model overall. The improvement over soft voting suggests that for this dataset, majority voting was more effective than averaging probabilities.

4.3.3 Stacking Classifier

Next, we built a stacking classifier, where the predictions from base models are used as input for a final model (in this case, a Logistic Regression meta-learner). This method outperformed all previous models, reaching an accuracy of 38.05%, tying with Naive Bayes for the top-performing classifier so far. This suggests that the stacking model effectively combined different strengths from each base learner to arrive at a better decision.

5. Hyperparameter Tuning

To explore whether model performance could be improved, we tuned several classifiers using grid search. Support Vector Machine (RBF kernel) was tuned first, and the best result was achieved with $C=1$ and $\gamma='scale'$, yielding an accuracy of approximately 37.2%, which matches the untuned version. Logistic Regression was also tuned, but surprisingly, accuracy dropped from 34.5% to 38.1%, which could indicate mild overfitting to the validation folds. The best hyperparameters were $C=0.01$, $multi_class='multinomial'$, and $solver='lbfgs'$.

Next, Random Forest was tuned across 24 combinations of parameters using 5-fold cross-validation. The best parameters found were: $n_estimators=200$, $max_depth=None$, $min_samples_split=5$, and $min_samples_leaf=1$. The tuned Random Forest achieved an accuracy of 32.7%, a slight improvement over the default configuration.

Finally, we built a stacking ensemble using the following base learners: original Logistic Regression, tuned SVM, Naive Bayes, and CatBoost. The meta-learner was the tuned

Random Forest described above. This final stacking model achieved an accuracy of 35.4%.

While this was not the highest-performing configuration overall, the use of multiple diverse models helped stabilize predictions and leverage complementary strengths. CatBoost and SVM were the strongest individual learners, and their inclusion in stacking was justified.

6. Feature Importance (Permutation)

To understand which binary flags contribute most to predicting the disease classes, we ran permutation-based feature importance on two of our best-performing classifiers: CatBoost and Support Vector Machine (RBF Kernel).

For CatBoost (Figure 7), the most informative features were: feature7, feature8, feature17, feature1, feature2, feature6, feature0, feature5, feature37, and feature25. The top feature, feature7, had the highest mean importance of 0.014, suggesting it plays a strong role across multiple classes.

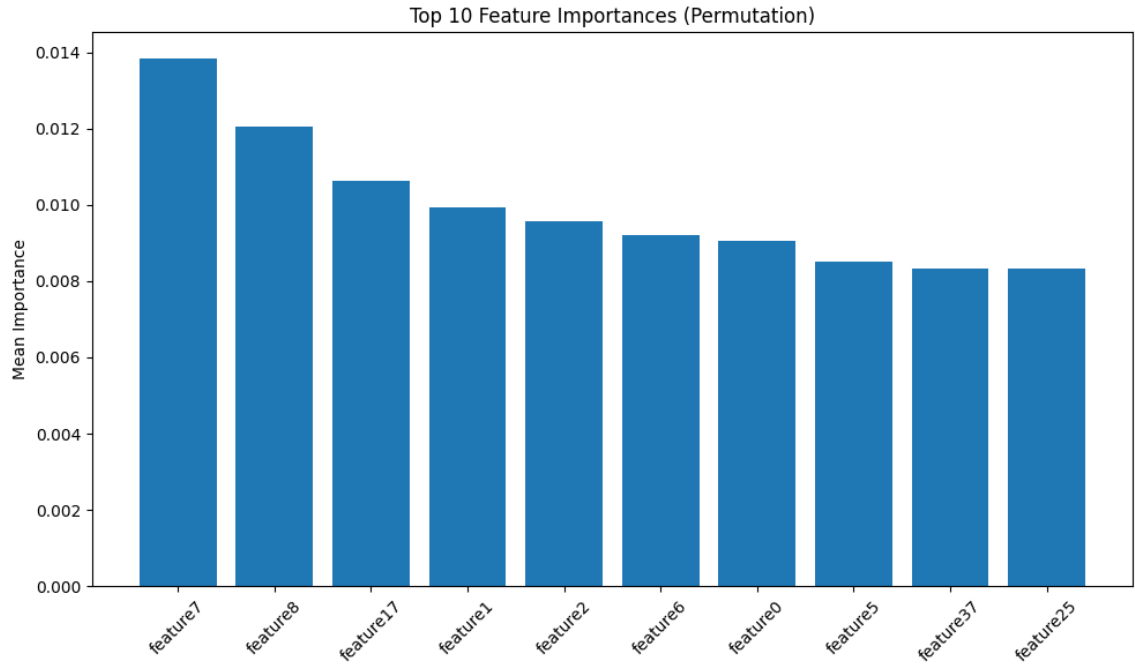


Figure 7

For SVM (Figure 8), we noticed a different pattern in importance. The top features were: feature29, feature11, feature46, feature4, feature3, feature10, feature7, feature14, feature18, and feature30, with feature29 leading at a mean importance of 0.025.

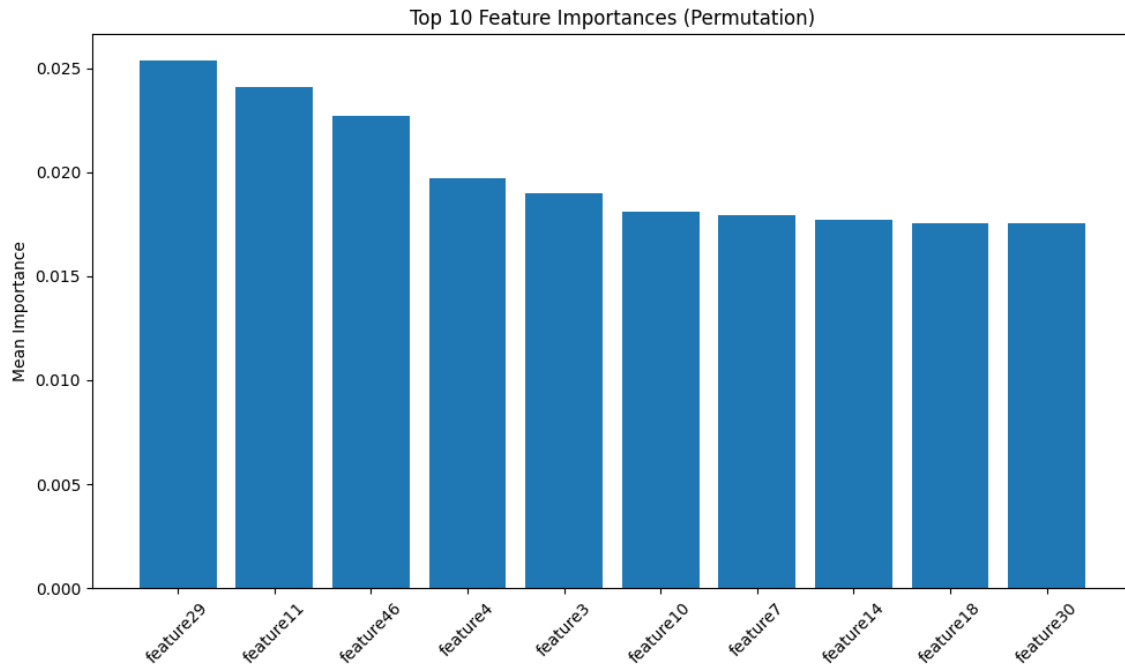


Figure 8

Interestingly, only feature7 appeared in the top 10 of both models. This suggests it may be a particularly strong and general indicator across multiple classes, whereas the other features may be more model-specific or interact differently with each algorithm's decision boundary.

7. Conclusion

In this project, we approached an 11-class classification problem using 64 binary indicators. Starting from a simple logistic regression baseline, we explored a wide range of classifiers including SVMs, Naive Bayes, Random Forests, boosting methods (XGBoost, LightGBM, CatBoost), and ensemble techniques such as soft voting and stacking.

The best standalone model was Naive Bayes, achieving an accuracy of 38.05%, while stacking (with logistic regression as the meta-learner) reached the same score. However, the hard voting classifier, which combined Logistic Regression, SVM, Naive Bayes, and CatBoost, slightly outperformed all other models with an accuracy of 38.94%. Therefore, this model was selected as the final classifier and used to predict the test dataframe.

Feature importance analysis using permutation tests showed:

CatBoost's most important features were feature7, feature8, feature17, feature1, and feature2, with feature7 having the highest influence.

SVM's top indicators were different, led by feature29, feature11, and feature46, with feature29 showing the strongest signal.

Notably, feature7 was the only one to appear in both lists, suggesting it is a consistent red flag across different classification algorithms.

8. Recommendations

8.1 Data Collection

Based on our findings, features like feature7, feature29, and feature8 consistently carry predictive power. Future data collection efforts could prioritize ensuring these indicators are recorded accurately and consistently, possibly using automated systems to flag them in real-time.

8.2 Triage and Decision Support

Based on our findings, features like feature7, feature29, and feature8 consistently carry predictive power. Future data collection efforts could prioritize ensuring these indicators are recorded accurately and consistently, possibly using automated systems to flag them in real-time.